```
c = a + b;
```

We accept that we're not arithmetically adding these two Strings. Rather, the plus [+] operator has been overloaded. "Overloading" is when something has a different meaning depending on its context. When we do this: c = a + b; it seems like we are really doing this: c = a.concat(b); In fact, secretly, we're using another class called a StringBuffer, which we'll get to a little later.

Take the following example, where we are concatenating a String a with an int i:

```
String a = "Ten ";
int i = 4;
String c ;
c = a + i;
```

Since i is being "added" to a String, the compiler knows it needs to convert i to a String also. The compiler creates a new String block just to hold the converted integer. Anytime you concatenate a String with another type, the other type is first converted into a String and then the two are concatenated. Really, this is done using method toString()which every object inherits from Object

```
c = a.concat( (String) b );
```

If you concatenate other primitive data types with a String, it has a similar effect:

```
String a = "";
boolean b = false;
String c ="Microsoft is dishonest=";
a = c + b;
Microsoft is dishonest=false
The += operator is also overloaded:
String c = "Microsoft ";
String b = "rules";
c += b;
{ c == "Microsoft rules" }
```

The following example is a crackerjack Certification question:

```
String a = "";
int b = 2;
int c = 3;
a = b + c;
```

This is a syntax error. To force the conversion to String, at least one of the